

Agent 定时执行平台需求文档

1. 背景

当前团队在使用 Codex、Claude Code、内部业务 Agent 等工具处理研发、运维、分析、巡检类任务。这类任务中有大量重复性工作，例如每日代码变更总结、定期检查错误日志、轮询 CI 状态、检查 PR Review 反馈、生成周报、定时执行质量分析等。

传统定时任务系统主要面向脚本、接口或批处理程序，任务逻辑通常由代码实现。而 Agent 类任务的特点是：任务内容可以通过自然语言描述，执行过程可能包含多轮推理、工具调用、文件修改、日志分析、报告生成和人工确认。因此需要一个面向 Agent 的定时执行平台，将“一次性对话式 Agent 能力”转化为“可长期运行、可观测、可治理的自动化能力”。

本需求参考 Codex App Automations 的产品思路，建设一个内部 Agent 定时执行平台。用户可以通过页面配置自然语言任务、调度周期、执行项目、session 策略、运行环境和通知规则，系统按计划唤起指定 Agent 执行，并沉淀执行日志、结果摘要、异常通知和可重跑记录。

2. 产品定位

本系统不是传统 cron 系统，而是 Agent Runtime 的周期性触发与治理平台。

一句话定义：

用户用自然语言定义周期性任务，系统按计划调度 Codex 等 Agent 在指定上下文中执行，并通过 session 管理、执行日志、结果判定和飞书通知，实现 Agent 任务的长期运行和可观测治理。

3. 目标

3.1 业务目标

1. 降低重复性研发、巡检、分析任务的人力成本。
2. 让 Codex 等 Agent 能够被稳定地周期性调用，而不是依赖人工手动打开会话。
3. 让 Agent 执行过程可追踪、结果可审计、异常可通知、失败可重跑。
4. 为后续多 Agent 自动化、AgentOps、Agent 平台化打基础。

3.2 产品目标

1. 支持用户通过页面创建 Agent 定时任务。
2. 支持自然语言描述任务内容。
3. 支持选择每次新建 session 或复用已有 session。
4. 支持查看每次执行记录、执行日志、最终结果和异常信息。
5. 支持配置飞书机器人，在异常、超时、高风险结果等场景下通知。
6. 支持手动触发、暂停、恢复、重跑任务。
7. 支持第一阶段接入 Codex，后续扩展到其他 Agent。

4. 非目标

第一阶段不做以下能力：

1. 不做复杂多 Agent 编排。
 2. 不做完整审批流系统。
 3. 不做低代码 workflow 画布。
 4. 不做跨团队 marketplace。
 5. 不做强多租户商业化能力。
 6. 不默认允许 Agent 自动发布线上变更。
 7. 不承诺 Agent 输出一定正确，系统只提供执行、记录、判定和治理能力。
-

5. 用户角色

5.1 任务创建者

通常是研发、架构师、运营、QA 或技术负责人。负责创建和维护 Agent 定时任务。

典型诉求：

- 配置一个每天自动执行的任务。
- 指定 Agent 要做什么。
- 指定是否复用历史上下文。
- 查看任务执行结果。
- 配置异常通知。

5.2 任务关注者

通常是团队成员、项目负责人或值班人员。主要关注任务结果和异常通知。

典型诉求：

- 通过飞书收到异常提醒。
- 点击链接查看执行详情。
- 判断是否需要人工介入。

5.3 平台管理员

负责 Agent 接入、权限、运行环境、任务安全策略和系统稳定性。

典型诉求：

- 控制哪些 Agent 可被调度。
 - 控制任务最大执行时间、并发数、权限范围。
 - 查看所有任务运行情况。
 - 排查执行失败原因。
-

6. 核心概念

6.1 Task：任务定义

Task 是用户配置的定时任务模板，描述“什么时候、用哪个 Agent、在哪个项目里、以什么上下文、执行什么任务”。

核心字段：

字段	说明
task_id	任务唯一 ID
task_name	任务名称
description	任务说明
agent_type	Agent 类型，例如 Codex
project	执行项目或工作目录
task_prompt	自然语言任务描述
schedule	定时配置
session_policy	session 策略
environment	执行环境
permission_policy	权限策略
notification_policy	通知策略
status	启用 / 暂停 / 删除
owner	任务负责人
created_at	创建时间
updated_at	更新时间

6.2 Run：执行实例

Run 是 Task 的一次具体执行记录。一个 Task 会产生多次 Run。

核心字段：

字段	说明
run_id	执行实例 ID
task_id	所属任务 ID
session_id	本次使用的 session

字段	说明
status	执行状态
trigger_type	定时触发 / 手动触发 / 重跑触发
started_at	开始时间
finished_at	结束时间
duration	执行耗时
result_summary	结果摘要
risk_level	风险等级
error_message	异常信息
artifact_links	产物链接
notify_status	通知状态

Run 状态包括：

状态	说明
PENDING	等待执行
RUNNING	执行中
SUCCESS	执行成功且有明确结果
NO_ACTION	执行成功但无事项需要处理
FAILED	执行失败
TIMEOUT	执行超时
CANCELLED	被人工取消
NEEDS_REVIEW	已完成但需要人工确认
PARTIAL_SUCCESS	部分成功
SYSTEM_ERROR	系统异常导致失败

6.3 Session: Agent 会话

Session 表示 Agent 的上下文容器。系统需要支持不同 session 策略，避免所有任务都只能从零开始或无限复用历史上下文。

第一阶段支持三种策略：

策略	说明	适用场景
always_new	每次执行新建 session	独立巡检、日报、周报

策略	说明	适用场景
reuse_fixed	固定复用一个 session	长期跟踪同一问题
reuse_last_success	复用最近一次成功执行的 session	连续任务，避免失败上下文污染

第二阶段支持：

策略	说明
inherit_summary	新建 session，但注入历史摘要
manual_resume	从指定 Run 对应 session 继续执行

默认策略建议为 always_new。对于需要持续上下文的任务，用户可以显式选择 reuse_fixed 或 reuse_last_success。

6.4 Agent Adapter: Agent 适配器

系统不直接绑定某一个 Agent，而是通过 Agent Adapter 抽象不同 Agent 的接入方式。

第一阶段只接入 Codex。

后续可扩展：

- Claude Code
- Gemini CLI
- OpenHands
- LangGraph Agent
- Browser Agent
- 内部业务 Agent

Agent Adapter 需要提供统一能力：

```
interface AgentAdapter {
  createSession(config): Promise<Session>;
  resumeSession(sessionId): Promise<Session>;
  run(input): AsyncIterator<AgentEvent>;
  cancel(runId): Promise<void>;
  getArtifacts(runId): Promise<Artifact[]>;
}
```

6.5 Evaluator: 结果判定器

Agent 说“完成了”不代表任务真的成功，因此系统需要独立的结果判定器。

Evaluator 负责根据 Agent 输出、运行状态、规则和结构化结果判断本次 Run 的最终状态。

判定依据包括：

- 1. Agent 进程是否正常退出。
- 2. 是否超时。
- 3. 是否产生有效最终输出。
- 4. 输出是否符合结构化格式。
- 5. 是否包含异常关键词。
- 6. 是否满足任务配置中的验收标准。
- 7. 是否产生预期产物，例如报告、PR、diff、文件、链接。
- 8. 是否需要人工 review。

7. 核心功能需求

7.1 任务创建

用户可以在页面创建一个 Agent 定时任务。

7.1.1 创建入口

页面提供「新建任务」按钮，点击后进入任务创建表单。

7.1.2 基础信息

用户需要填写：

字段	是否必填	说明
任务名称	是	用于任务列表展示
任务描述	否	补充说明
负责人	是	默认当前用户
所属项目	是	选择执行项目
Agent 类型	是	第一阶段默认 Codex

7.1.3 任务内容

用户通过自然语言描述任务。

页面提供一个大文本框，标题为「你希望 Agent 定时做什么？」

推荐用户按照以下结构填写：

目标：你要完成什么任务？
上下文：任务要关注哪些项目、文件、日志、系统、数据？
约束：哪些事情不能做？是否允许改代码？是否允许创建 PR？

完成标准：什么情况下算完成？需要输出什么？
异常标准：什么情况下需要告警？

示例：

每天上午 10 点检查 pinme 项目最近 24 小时的错误日志。
重点关注 5xx、登录失败、支付失败、连接超时。
如果同类错误超过 10 次，分析可能原因并生成修复建议。
不要直接修改代码，不要提交 PR，只输出报告。
如果风险等级为 high，通知飞书群。

7.2 调度配置

用户可以配置任务执行周期。

第一阶段支持：

1. 每小时。
2. 每天固定时间。
3. 每周固定日期和时间。
4. 自定义 cron 表达式。
5. 手动触发，不配置周期。

字段：

字段	说明
schedule_type	hourly / daily / weekly / cron / manual
cron_expression	自定义 cron
timezone	时区，默认 Asia/Shanghai
next_run_at	下次执行时间
enabled	是否启用

系统需要在页面展示：

- 当前调度规则。
- 下次执行时间。
- 最近一次执行时间。
- 最近一次执行状态。

7.3 Session 策略配置

用户创建任务时需要选择 session 策略。

第一阶段提供：

7.3.1 每次新建 session

每次执行都是独立上下文。

适合：

- 日报。
- 周报。
- 巡检。
- 日志分析。
- 代码变更总结。

7.3.2 固定复用 session

所有执行都进入同一个 session。

适合：

- 持续跟踪一个问题。
- 长周期 research。
- 等待外部状态变化。
- 持续处理同一个 PR。

7.3.3 复用上一次成功 session

只有上一次成功执行的 session 会被复用。如果上一次失败，则继续使用更早的成功 session，避免失败上下文污染。

适合：

- 需要上下文连续性，但希望降低污染风险的任务。

7.4 执行环境配置

用户可以配置 Agent 在哪里执行。

第一阶段以代码项目为主，支持：

配置	说明
项目目录	Agent 执行的工作目录
执行模式	local / isolated_workspace
分支	默认当前分支，可选指定分支
环境变量	可选，敏感信息加密存储

配置	说明
超时时间	默认 30 分钟
最大步数	防止 Agent 无限循环
模型配置	默认值 / 指定模型
推理强度	默认值 / low / medium / high

local 模式

Agent 直接在项目目录执行。风险较高，可能影响当前工作区。

isolated_workspace 模式

系统为每次 Run 创建独立工作区，类似 worktree/sandbox，用于隔离 Agent 修改。

第一阶段建议默认使用 isolated_workspace。

7.5 权限与安全配置

由于 Agent 是无人值守执行，必须限制默认权限。

第一阶段支持任务级权限：

权限	默认值	说明
只读文件	开启	允许读取项目文件
写入工作区	关闭	是否允许修改文件
执行命令	受限	只允许白名单命令
访问网络	关闭	是否允许访问外部网络
创建 PR	关闭	是否允许创建 PR
发送消息	关闭	是否允许主动发通知外的消息
访问生产环境	关闭	默认禁止

权限策略分为三档：

ReadOnly

只允许读取文件、日志、配置，不能修改任何内容。

WorkspaceWrite

允许在隔离工作区修改文件、生成 diff、生成报告，但不能直接影响主目录。

FullAccess

允许更高权限操作，例如访问网络、运行更多命令、创建 PR。该模式需要管理员授权。

第一阶段默认使用 ReadOnly 或 WorkspaceWrite，不建议默认开放 FullAccess。

7.6 手动测试任务

在任务正式启用前，系统提供「测试运行」能力。

用户点击「测试运行」后，系统立即创建一次 Run，但不影响正式调度。

测试运行的目标：

1. 验证 prompt 是否清楚。
2. 验证 Agent 是否能访问所需上下文。
3. 验证权限是否足够。
4. 验证输出是否符合预期。
5. 验证通知规则是否正确。

测试完成后，页面提示：

- 测试是否成功。
- Agent 输出摘要。
- 是否发现权限问题。
- 是否建议调整 prompt。
- 是否可以启用任务。

7.7 执行日志

每次 Run 都需要记录完整执行过程。

日志类型包括：

日志类型	说明
system_event	系统事件，例如创建 Run、分配 Runner、超时
agent_message	Agent 输出
tool_call	工具调用
command_output	命令执行输出
file_change	文件变更
artifact	生成产物
evaluator_result	结果判定

日志类型	说明
notification_event	通知事件

Run 详情页需要展示：

1. 执行基本信息。
2. 最终状态。
3. 结果摘要。
4. 风险等级。
5. Agent 对话过程。
6. 工具调用记录。
7. 命令输出。
8. 文件 diff 或产物。
9. Evaluator 判定原因。
10. 飞书通知记录。

7.8 结果输出

要求 Agent 最终输出结构化结果。

推荐格式：

```
{
  "status": "success | no_action | failed | needs_review | partial_success",
  "summary": "本次执行摘要",
  "findings": [
    {
      "title": "发现项标题",
      "severity": "low | medium | high",
      "description": "发现项说明",
      "evidence": ["证据 1", "证据 2"],
      "suggestion": "处理建议"
    }
  ],
  "artifacts": [
    {
      "type": "report | diff | pr | file | link",
      "name": "产物名称",
      "url": "产物链接"
    }
  ],
  "risk_level": "none | low | medium | high",
  "need_notify": true,
  "need_human_review": false
}
```

系统需要校验结构化结果。如果 Agent 没有输出合法结构，Evaluator 应标记为 PARTIAL_SUCCESS 或 FAILED，并记录原因。

7.9 飞书机器人通知

系统支持为每个任务配置飞书机器人通知。

7.9.1 通知配置

字段：

字段	说明
webhook_url	飞书机器人 webhook
secret	飞书机器人签名密钥
notify_on_failed	失败时通知
notify_on_timeout	超时时通知
notify_on_high_risk	高风险时通知
notify_on_needs_review	需要人工确认时通知
notify_on_every_run	每次执行都通知
consecutive_failure_threshold	连续失败 N 次通知

7.9.2 通知内容

飞书消息需要包含：

任务名称：每日错误日志巡检

执行状态：FAILED

风险等级：HIGH

执行时间：2026-06-18 10:00

耗时：12 分钟

摘要：登录接口 5xx 在过去 24 小时内明显增加

可能原因：数据库连接池耗尽

建议动作：查看连接池配置与慢查询

详情链接：点击查看 Run 日志

第二阶段支持飞书卡片按钮：

- 查看详情。
 - 重新执行。
 - 暂停任务。
 - 标记已处理。
-

7.10 任务列表

任务列表展示所有任务。

字段：

字段	说明
任务名称	可点击进入详情
状态	启用 / 暂停
Agent	Codex
调度规则	每天 10:00
Session 策略	新建 / 复用
最近执行状态	SUCCESS / FAILED
最近执行时间	时间
下次执行时间	时间
负责人	用户
操作	执行 / 暂停 / 编辑 / 删除

支持筛选：

- 任务状态。
- 最近执行状态。
- 负责人。
- Agent 类型。
- 项目。
- 是否有异常。

7.11 任务详情页

任务详情页包含：

1. 基础信息。
2. 任务 prompt。
3. 调度配置。
4. Session 策略。
5. 执行环境。
6. 权限配置。
7. 通知配置。
8. 执行历史。
9. 任务统计。

任务统计包括：

- 总执行次数。
- 成功次数。
- 失败次数。
- 超时次数。
- 最近 7 天成功率。
- 平均执行耗时。
- 最近一次异常原因。

7.12 Run 详情页

Run 详情页用于排查一次执行。

页面模块：

1. Run 基础信息。
2. 最终结果摘要。
3. Agent 执行日志。
4. 工具调用。
5. 文件变更。
6. 产物列表。
7. Evaluator 判定。
8. 通知记录。
9. 操作区。

操作区支持：

- 重新执行。
- 使用相同 session 重跑。
- 使用新 session 重跑。
- 取消执行。
- 标记已处理。
- 复制结果摘要。

7.13 重跑能力

用户可以对失败或历史 Run 执行重跑。

重跑方式：

方式	说明
使用原配置重跑	使用当前 Task 配置重新执行
使用原 session 重跑	延续当时 session
使用新 session 重跑	完全重新开始

方式	说明
修改 prompt 后重跑	临时调整 prompt，不影响 Task 默认配置

重跑需要生成新的 Run，不覆盖历史 Run。

7.14 暂停与恢复

用户可以暂停任务。

暂停后：

1. Scheduler 不再触发新 Run。
2. 已经运行中的 Run 不受影响。
3. 用户可以手动触发一次执行。
4. 用户可以恢复任务。

7.15 并发与防重

系统需要防止同一个任务重复执行。

任务级并发策略：

策略	说明
skip_if_running	如果上一次还在执行，本次跳过
queue_if_running	如果上一次还在执行，本次排队
allow_parallel	允许并发执行

第一阶段默认 skip_if_running。

8. 典型使用场景

8.1 每日代码变更简报

每天上午 10 点，Agent 检查指定代码仓库最近 24 小时提交记录，按模块总结变化，输出 Markdown 简报。如果发现高风险改动，通知飞书群。

8.2 每小时错误日志巡检

每小时检查服务日志。如果发现 5xx、超时、登录失败、支付失败等异常明显增加，则生成分析报告并通知飞书群。

8.3 定期检查 PR Review 反馈

每 30 分钟检查指定 PR 是否有新 review comment。如果有，Agent 分析反馈内容，给出修改建议。需要修改代码时进入人工 review 状态。

8.4 每周生成项目周报

每周五下午，Agent 根据 Git 提交、Issue、PR、日报等信息生成项目周报，输出给负责人。

8.5 定期扫描重复问题并沉淀 skill

每天扫描 Agent 历史执行记录，如果发现重复失败模式，生成优化建议，提示是否沉淀为 reusable skill。

9. 页面原型说明

9.1 新建任务页

页面结构：

1. 基础信息。
2. 选择 Agent。
3. 选择项目。
4. 编写任务 prompt。
5. 配置调度周期。
6. 配置 session 策略。
7. 配置运行环境。
8. 配置权限。
9. 配置通知。
10. 测试运行。
11. 保存并启用。

9.2 任务列表页

展示所有任务，并突出最近异常任务。

9.3 任务详情页

展示任务配置和执行历史。

9.4 Run 详情页

展示一次执行的完整过程和产物。

9.5 Triage 页

参考 inbox 思路，集中展示需要关注的运行结果。

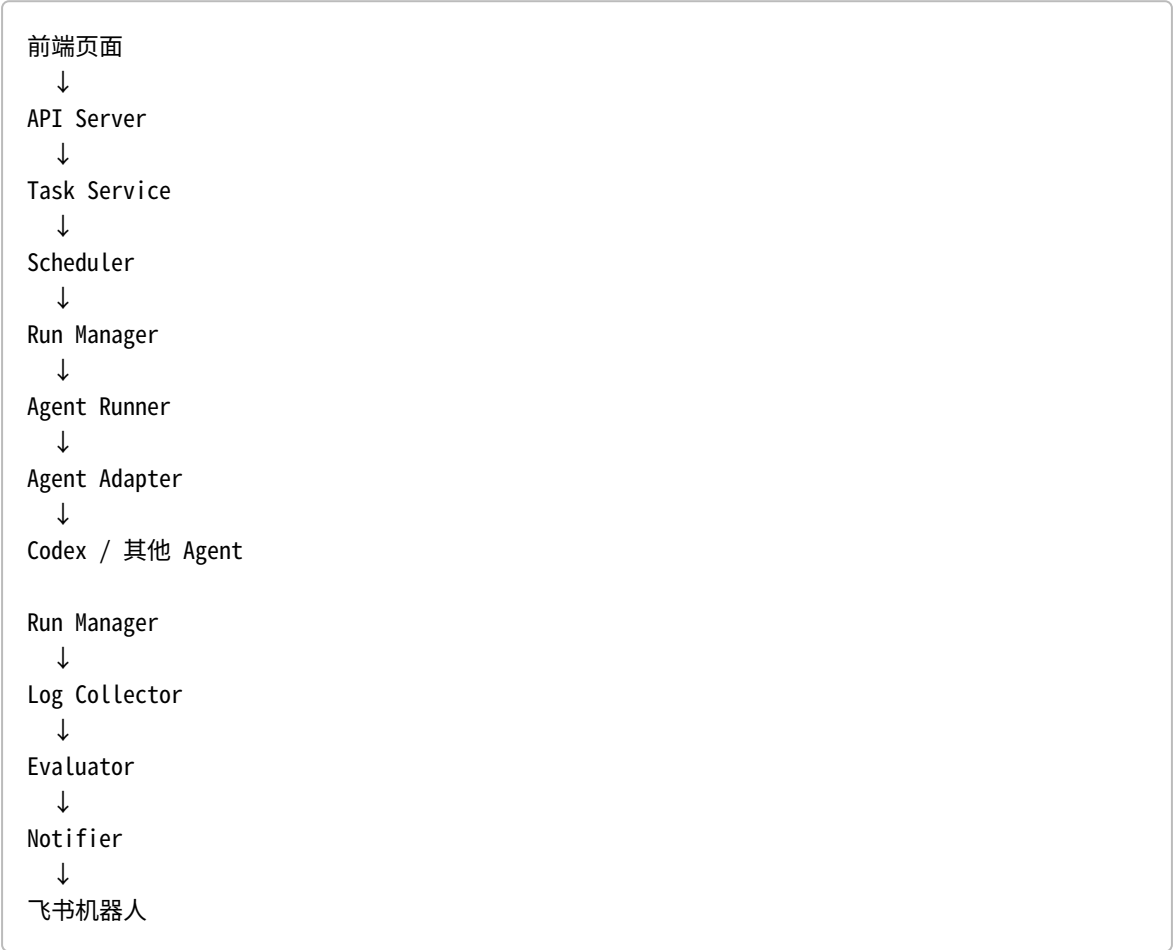
进入 Triage 的条件：

- 1. 执行失败。
- 2. 执行超时。
- 3. 有 high 风险发现。
- 4. 需要人工 review。
- 5. Agent 明确报告有发现项。
- 6. 连续失败超过阈值。

无事项的成功执行可以自动归档，但仍可在执行历史中查询。

10. 系统架构

整体架构：



核心模块：

模块	职责
Web Console	任务配置、执行记录、日志查看
API Server	对外提供任务、Run、配置 API

模块	职责
Scheduler	定时触发任务
Run Manager	管理执行生命周期
Agent Runner	启动和监控 Agent
Codex Adapter	适配 Codex 执行
Session Manager	管理 session 创建、复用、摘要
Log Collector	收集流式日志
Evaluator	判定执行结果
Notifier	推送飞书通知
Artifact Store	存储报告、diff、文件、链接
Permission Manager	管理权限和执行策略

11. 数据模型初版

11.1 task 表

```
CREATE TABLE agent_task (
  id BIGINT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  agent_type VARCHAR(64) NOT NULL,
  project_id BIGINT,
  prompt TEXT NOT NULL,
  schedule_type VARCHAR(32),
  cron_expression VARCHAR(128),
  timezone VARCHAR(64),
  session_policy VARCHAR(64),
  environment_config JSON,
  permission_config JSON,
  notification_config JSON,
  status VARCHAR(32),
  owner_id BIGINT,
  next_run_at TIMESTAMP,
  last_run_at TIMESTAMP,
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);
```

11.2 run 表

```
CREATE TABLE agent_task_run (  
  id BIGINT PRIMARY KEY,  
  task_id BIGINT NOT NULL,  
  session_id VARCHAR(255),  
  trigger_type VARCHAR(64),  
  status VARCHAR(64),  
  risk_level VARCHAR(32),  
  result_summary TEXT,  
  structured_result JSON,  
  error_message TEXT,  
  started_at TIMESTAMP,  
  finished_at TIMESTAMP,  
  duration_ms BIGINT,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

11.3 log 表

```
CREATE TABLE agent_task_run_log (  
  id BIGINT PRIMARY KEY,  
  run_id BIGINT NOT NULL,  
  log_type VARCHAR(64),  
  content TEXT,  
  metadata JSON,  
  created_at TIMESTAMP  
);
```

11.4 artifact 表

```
CREATE TABLE agent_task_artifact (  
  id BIGINT PRIMARY KEY,  
  run_id BIGINT NOT NULL,  
  artifact_type VARCHAR(64),  
  name VARCHAR(255),  
  url TEXT,  
  metadata JSON,  
  created_at TIMESTAMP  
);
```

11.5 session 表

```
CREATE TABLE agent_session (  
  id VARCHAR(255) PRIMARY KEY,  
  agent_type VARCHAR(64),
```

```
task_id BIGINT,  
status VARCHAR(64),  
summary TEXT,  
metadata JSON,  
created_at TIMESTAMP,  
updated_at TIMESTAMP  
);
```

12. API 初版

12.1 创建任务

```
POST /api/agent-tasks
```

12.2 编辑任务

```
PUT /api/agent-tasks/{task_id}
```

12.3 查询任务列表

```
GET /api/agent-tasks
```

12.4 查询任务详情

```
GET /api/agent-tasks/{task_id}
```

12.5 启用任务

```
POST /api/agent-tasks/{task_id}/enable
```

12.6 暂停任务

```
POST /api/agent-tasks/{task_id}/pause
```

12.7 手动触发任务

```
POST /api/agent-tasks/{task_id}/runs
```

12.8 查询执行历史

```
GET /api/agent-tasks/{task_id}/runs
```

12.9 查询 Run 详情

```
GET /api/agent-task-runs/{run_id}
```

12.10 查询 Run 日志

```
GET /api/agent-task-runs/{run_id}/logs
```

12.11 重跑

```
POST /api/agent-task-runs/{run_id}/rerun
```

12.12 取消执行

```
POST /api/agent-task-runs/{run_id}/cancel
```

13. Codex Adapter 第一阶段实现建议

第一阶段可以先以 Codex CLI 非交互模式作为执行入口。

执行方式：

```
codex exec "<task_prompt>"
```

系统需要包装以下能力：

1. 注入工作目录。
2. 注入任务 prompt。
3. 注入权限配置。
4. 注入超时时间。
5. 收集 stdout 作为最终结果。
6. 收集 stderr 作为过程日志。
7. 记录退出码。
8. 保存产物。
9. 解析结构化结果。

对于需要 session 复用的任务，后续可以引入 Codex MCP Server 或 App Server 模式，使系统能够显式管理 thread/session。

第一阶段建议优先支持：

- always_new。
- isolated_workspace。
- 结构化输出。
- 执行日志。
- 飞书异常通知。

14. Prompt 模板

系统在用户 prompt 外层增加统一包装，保证 Agent 输出可判定。

模板：

你是一个被定时任务系统调度的 Agent。

本次任务如下：

{{user_task_prompt}}

执行约束：

{{permission_policy}}

当前项目：

{{project_context}}

本次执行要求：

1. 严格按照任务目标执行。
2. 不要做任务之外的事情。
3. 如果权限不足，明确说明。
4. 如果没有发现问题，返回 no_action。
5. 如果发现风险，说明证据和建议。
6. 最终必须输出 JSON，格式如下：

```
{
  "status": "success | no_action | failed | needs_review | partial_success",
  "summary": "本次执行摘要",
  "findings": [],
  "artifacts": [],
  "risk_level": "none | low | medium | high",
  "need_notify": false,
  "need_human_review": false
}
```

15. 通知规则

系统最终是否通知，由以下条件决定：

```
should_notify =  
    notify_on_every_run  
    OR run.status in [FAILED, TIMEOUT, SYSTEM_ERROR]  
    OR structured_result.risk_level = high  
    OR structured_result.need_notify = true  
    OR structured_result.need_human_review = true  
    OR consecutive_failures >= threshold
```

16. 成功指标

第一阶段上线后关注以下指标：

指标	目标
任务创建成功率	> 95%
定时触发成功率	> 99%
Run 日志完整率	> 95%
飞书通知成功率	> 99%
任务误告警率	持续下降
连续失败可定位率	> 90%
用户手动重跑成功率	> 90%

17. MVP 范围

第一阶段必须完成：

1. 任务创建、编辑、暂停、删除。
2. cron / daily / weekly 调度。
3. Codex Agent 接入。
4. 每次新建 session。
5. 复用固定 session 的配置入口。
6. Run 执行记录。
7. Run 日志查看。
8. 结构化结果输出。
9. 简单 Evaluator。
10. 飞书机器人通知。
11. 手动触发。
12. 失败重跑。
13. Triage 异常列表。

第一阶段可以暂缓：

1. 多 Agent 接入。

2. 复杂审批流。
 3. 飞书卡片操作按钮。
 4. 自动创建 PR。
 5. 长期记忆。
 6. session summary。
 7. 多租户隔离。
 8. 细粒度命令白名单管理页面。
-

18. 风险与解决方案

18.1 Agent 执行结果不稳定

解决方案：

- 强制结构化输出。
- 增加 Evaluator。
- 上线前提供测试运行。
- 关键任务默认 needs_review。

18.2 session 上下文污染

解决方案：

- 默认每次新建 session。
- 失败 session 不参与复用。
- 后续支持历史摘要继承，而不是无限复用完整上下文。

18.3 Agent 无人值守执行带来安全风险

解决方案：

- 默认只读。
- 默认隔离工作区。
- 高权限需要管理员授权。
- 敏感操作进入人工 review。
- 保留完整审计日志。

18.4 日志过多难以阅读

解决方案：

- 日志分层展示。
- 默认展示摘要。
- 支持展开原始日志。
- 支持按 tool_call、error、artifact 过滤。

18.5 飞书告警噪音过多

解决方案：

- 支持通知规则。
 - 支持连续失败合并。
 - no_action 自动归档。
 - 支持 Triage 聚合查看。
-

19. 里程碑

M1：基础任务调度

完成任务 CRUD、调度触发、Run 记录。

M2：Codex 执行接入

完成 Codex Adapter、执行日志采集、超时控制。

M3：结果判定与日志页面

完成结构化结果解析、Evaluator、Run 详情页。

M4：飞书通知

完成异常通知、通知配置、通知记录。

M5：Triage 与重跑

完成异常收件箱、手动重跑、session 策略入口。

20. 总结

该平台的核心价值不是“定时执行”，而是让 Agent 具备稳定的周期性工作能力。它将自然语言任务、调度、session、运行环境、日志、结果判定和通知统一起来，使 Codex 等 Agent 从一次性助手升级为可长期运行的数字员工。

第一阶段应聚焦 Codex + 定时调度 + 日志 + 结构化结果 + 飞书通知，先验证核心闭环。后续再扩展多 Agent、审批流、长期记忆、飞书卡片、PR 自动化和企业治理能力。